

CHAPITRE 4 : SOUS-REQUETES, CTEs ET VUES

4.1 Les sous-requêtes.

Une sous-requête est une requête SQL écrite à l'intérieur d'une autre requête. Elle permet d'utiliser le résultat d'une requête pour effectuer une autre recherche.

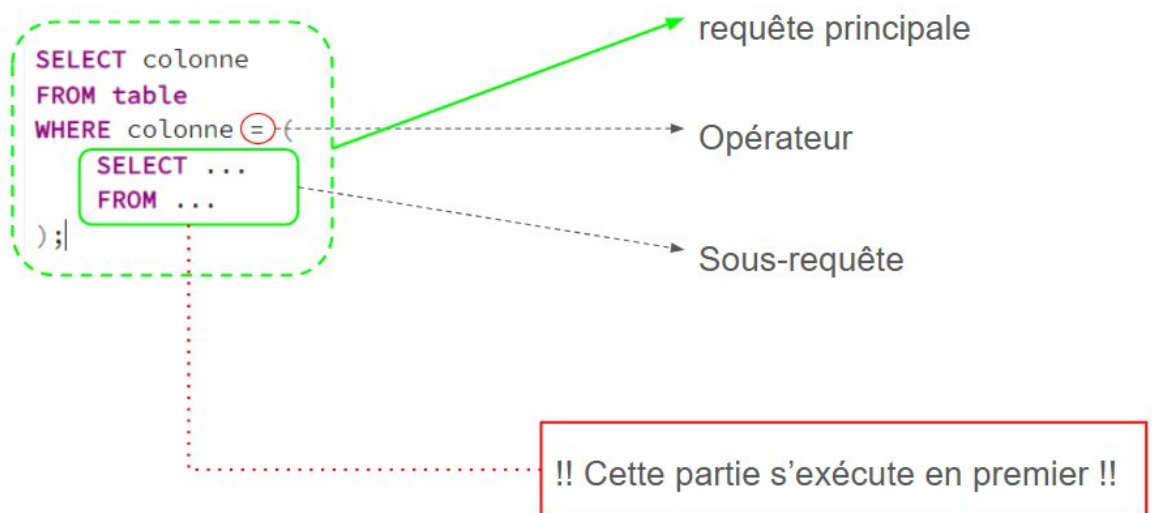
Une sous-requête sert à récupérer une information qui sera utilisée par la requête principale.

Elle peut être dans le WHERE, le FROM, ou le SELECT.

Exemple :

(avec WHERE)

Sous-requête avec WHERE



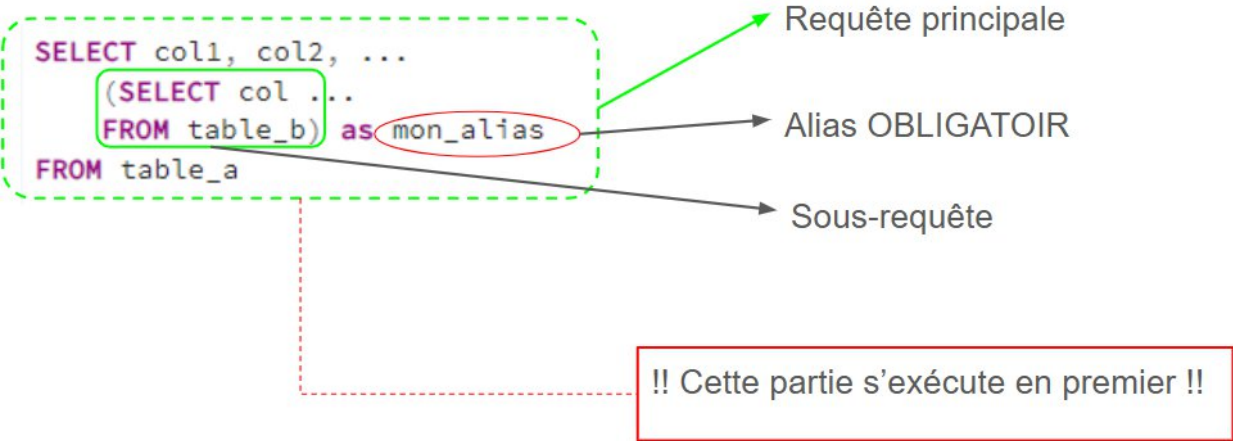
```
-- Sous-requete dans WHERE : produits plus chers que la moyenne
SELECT nom, prix
FROM produits
WHERE prix > (SELECT AVG(prix) FROM produits);
```

NB : (Le calcul de la moyenne de la sous-requête est calculé avant le reste)

Je récupère le nom, et le prix du produit de la table produits lorsque le prix est supérieur à la moyenne des prix de la table produit (calculé dans la sous-requête).

(Avec SELECT)

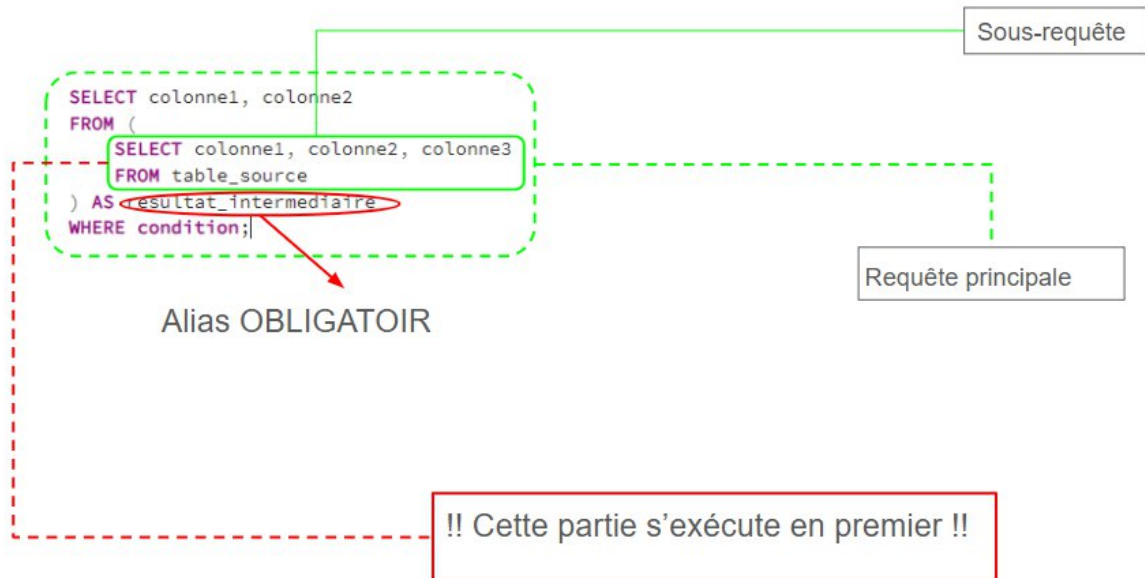
Sous-requête avec SELECT



DANS LE SELECT LA SOUS-REQUETE NE RETOURNE QU'UNE SEULE VALEUR PAR LIGNE

Renvoie une seul valeur	Renvoie plusieurs lignes	Renvoie plusieurs colonnes												
<pre>SELECT c.nom, (SELECT COUNT(*) FROM commandes co WHERE co.client_id = c.client_id) AS nb_commandes FROM clients c;</pre>	<pre>SELECT c.nom, (SELECT total FROM commandes co WHERE co.client_id = c.client_id) AS nb_commandes FROM clients c;</pre>	<pre>SELECT c.nom, (SELECT total, commande_id FROM commandes co WHERE co.client_id = c.client_id) AS nb_commandes FROM clients c;</pre>												
<table><tr><th></th><th>nom character varying (100)</th><th>nb_commandes bigint</th></tr><tr><td>1</td><td>Alice Martin</td><td>3</td></tr><tr><td>2</td><td>Bob Dupont</td><td>2</td></tr><tr><td>3</td><td>Claire Moreau</td><td>2</td></tr></table>		nom character varying (100)	nb_commandes bigint	1	Alice Martin	3	2	Bob Dupont	2	3	Claire Moreau	2	ERREUR: plus d'une ligne renvoyée par une sous-requête utilisée comme une expression ERREUR: plus d'une ligne renvoyée par une sous-requête utilisée comme une expression SQL state: 21000	ERREUR: la sous-requête doit renvoyer une seule colonne SQL state: 42601 Character: 23
	nom character varying (100)	nb_commandes bigint												
1	Alice Martin	3												
2	Bob Dupont	2												
3	Claire Moreau	2												

(Avec FROM)



```

-- Sous-requete dans FROM (table derivee)
SELECT client_id, nb_commandes
FROM (
  SELECT client_id, COUNT(*) AS nb_commandes FROM
  commandes
  GROUP BY client_id ) AS
stats_clients
WHERE nb_commandes > 5;

```

Etape 1 - la sous-requête : compter le nombre de commandes pour chaque clients

```

SELECT client_id, COUNT(*) AS nb_commandes
FROM commandes
GROUP BY client_id;

```

Etape 2 – Considère ce résultat comme une table.

stats_clients

Etape 3 – La requête principale

```

SELECT client_id, nb_commandes
FROM stats_clients
WHERE nb_commandes > 5;

```

Sous-requête corrélée (avancé) :

Sous-requête simple	Sous-requête corrélée
<pre>Select nom, prix From produits Where prix > (Select avg(prix) From produits);</pre> _ S'exécute une seule fois _ indépendant de la requête principale	<pre>Select p1.nom, p1.prix From produits p1 Where p1.prix > (Select avg(p2.prix) From produits p2 Where p2.categorie = p1.categorie);</pre> _ S'exécute une fois par ligne (pour toutes les lignes de la table)

Exemple de sous-requête corrélée :

Récupérer les produit qui ont un prix supérieur au pris moyen de la catégorie à laquelle ils appartiennent

```
select p1.nom, p1.prix
from produits p1
where p1.prix > (
    select avg(p2.prix)
    from produits p2
    where p1.categorie = p2.categorie
);
```

-- alias p1 = requête principale

-- p1 reference depuis la sous-requête

À l'inverse, d'une sous-requête corrélée, une sous-requête non corrélée s'exécute indépendamment de la requête externe. Elle s'exécute une seule fois, renvoie un ensemble de résultats ou une valeur, et la requête externe réutilise ce résultat sans relancer la sous-requête pour chaque ligne.

Autre exemple : employés gagnant au-dessus de la moyenne de leur service

Supposons que vous ayez une table « employees » avec les salaires et les identifiants de service. Vous souhaitez trouver les employés qui gagnent plus que la moyenne de leur service.

Vous pouvez utiliser la requête ci-dessous, où :

```

SELECT
    e.employee_id,
    e.employee_name,
    e.salary,
    e.departement_id
FROM employees e
WHERE e.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.departement_id = e.departement_id
);

```

Pour aller plus loin :

<https://www.datacamp.com/fr/tutorial/correlated-subquery>

4.2 COMMON TABLE EXPRESSION CTEs

Une CTE est une sous-requête nommée définie avant la requête principale avec le mot clé WITH. Elle se comporte comme une table temporaire qu'on peut réutiliser dans la requête principale.

La CTE résout 2 problèmes des sous-requêtes dans le FROM :

- Le code est plus lisible, on donne un nom parlant à chaque étape
- On peut réutiliser la même CTE à plusieurs reprises dans la requête principale
- On peut enchaîner plusieurs CTEs pour découper un problème complexe

Syntaxe de base :

```

-- syntaxe de base CTE
WITH nom_cte AS (
    -- requete de la CTE
    SELECT ...
    FROM ...
    WHERE ...
)
-- requete principale
SELECT *
FROM nom_cte
WHERE ... ;|

```

Comparaison CTE et sub-query sur un exemple simple :

Sous-requête simple	CTE
<pre>SELECT client_id, ca_total, nom FROM (SELECT cl.client_id, cl.nom, ROUND(SUM(co.total), 2) AS ca_total FROM commandes co INNER JOIN clients cl ON co.client_id = cl.client_id GROUP BY cl.client_id, cl.nom) AS stats WHERE ca_total > 100;</pre>	<pre>WITH ca_par_client AS(SELECT cl.client_id, cl.nom, round(sum(co.total),2) as ca_total from commandes co inner join clients cl on co.client_id = cl.client_id group by cl.client, cl.nom) select nom, ca_total from ca_par_client where ca_total > 100 order by ca_total desc</pre>

Il est possible d'enchaîner plusieurs CTE.

Syntaxe :

```
WITH premiere_cte AS (
  -- première requête
  SELECT ...
  FROM ...
),
deuxieme_cte AS (
  -- utilise la première CTE
  SELECT ...
  FROM premiere_cte
),
troisieme_cte AS (
  -- utilise la deuxième CTE
  SELECT ...
  FROM deuxieme_cte
)

SELECT *
FROM troisieme_cte;
```

	Sous-requete dans FROM	CTE (WITH)
Lisibilité	Moins lisible (imbriquée)	Plus lisible (nommée)
Reutilisation	Non (à réécrire)	Oui (dans la même requête)
Cas simple	Convient bien	Un peu verbeux
Cas complexe	Difficile à lire	Ideal
Preference	—	A privilégier en general

4.3 LES VUES.

Une vue est une requête sauvegardée dans la base de données sous un nom. Elle se comporte exactement comme une table, on peut faire des SELECT dessus, la joindre avec d'autres tables, la filtrer... Mais elle ne stocke pas de données : elle les recalcule à chaque appel. C'est utile en outre quand une même requête complexe est utilisée souvent par plusieurs personnes.

```
-- Creer une vue
CREATE VIEW nom_vue AS
SELECT ...;
-- Creer ou remplacer une vue existante
CREATE OR REPLACE VIEW nom_vue AS
SELECT ...;
-- Utiliser la vue comme une table
SELECT * FROM nom_vue;
SELECT * FROM nom_vue WHERE ...;
-- Supprimer une vue
DROP VIEW nom_vue;
DROP VIEW IF EXISTS nom_vue;
```

SYNTHESE

	Sous-requete	CTE (WITH)	Vue (VIEW)
Duree de vie	Le temps de la requete	Le temps de la requete	Permanente en base
Reutilisable	Non	Dans la meme requete	Partout et par tous
Stocke les data	Non	Non	Non (recalcule)
Modifiable	—	—	Oui (OR REPLACE)
Ideal pour	Filtre ponctuel	Requete complexe	Requete souvent utilisee

Sous-requête simple	Une requête dans une requête. La sous-requête s'exécute en premier . Elle peut-être dans le WHERE (retourne une valeur ou une liste) dans le FROM (retourne une table => alias OBLIGATOIRE), ou dans le SELECT (retourne une valeur par ligne)
Sous-requête corrélée	Comme une sous-requête simple mais elle fait référence à une requête principale grâce à l'alias. Elle s'exécute une fois par ligne. Notion à connaître mais privilégier les jointures ou les CTE.
CTE (WITH ...)	Une sous-requête nommée définie avant la requête principale. Plus

AS)	lisible qu'une sous-requête dans le FROM. On peut enchaîner les CTEs séparées pas des ','. A privilégier pour les requêtes complexes.
Vue (VIEW)	Une requête sauvegardée de façon permanente dans la base de donnée. Se comporte comme une table. Ne stocke pas les données, elle les recalcule à chaque appel. Idéale pour les requêtes souvent réutilisées ou pour simplifier l'accès aux données.

Cheat Sheet Syntaxe :

```
-- SOUS-REQUETE dans WHERE
SELECT * FROM table WHERE col > (SELECT AVG(col) FROM table);
SELECT * FROM table WHERE col IN (SELECT col FROM autre_table);
SELECT * FROM table WHERE col NOT IN (SELECT col FROM autre_table);
-- SOUS-REQUETE dans FROM (alias obligatoire)
SELECT * FROM (SELECT ...) AS alias WHERE ...;
-- CTE simple
WITH ma_cte AS (SELECT ...)
SELECT * FROM ma_cte WHERE ...;
-- Plusieurs CTEs
WITH cte1 AS (SELECT ...), cte2 AS (SELECT ...)
SELECT * FROM cte1 JOIN cte2 ON ...;
-- VUE
CREATE VIEW ma_vue AS SELECT ...;
SELECT * FROM ma_vue; -- utilisation
CREATE OR REPLACE VIEW ma_vue AS SELECT ...; -- modification
DROP VIEW IF EXISTS ma_vue; -- suppression
```

CHAPITRE 5 : FONCTIONS SQL UTILES

5.1 FONCTIONS SUR LES CHAINES DE CARACTERE

COMMANDE	DESCRIPTION
UPPER	MAJUSCULES
LOWER	MINUSCULE
LENGTH	LONGUEUR
TRIM	SUPPRESSION DES ESPACES
LTRIM	SUPPRESSION ESPACES (GAUCHE)
CONCAT	CONCATECNER 2 COLONNES AVEC UN CARACTERE
	CONCATENATION METHODE 2
SUBSTRING	EXTRACTION DE CARACTERES
REPLACE	REMPLACEMENT DE CARACTERES
SPLIT_PART	RECUPERER UNE PARTIE DE LA COLONNE
LEFT	N PREMIERS CARACTERES
RIGHT	N DERNIER CARACTERES


```

SELECT UPPER(col) FROM table_1; -- MAJUSCULES
SELECT LOWER(col) FROM table_1; -- minuscules
SELECT LENGTH(col) FROM table_1; -- longueur (attention l'espace compte comme 1 caractère)
SELECT TRIM(' caractères avec espaces '); -- supprime espaces a droite et a gauche
SELECT LTRIM(' caractères avec espaces '); -- espaces a gauche
SELECT CONCAT(col1, ' - ', col2) FROM table_1; -- concatener 2 colonnes avec un caractère entre les 2
SELECT col1 || ' (' || col2 || ' )' FROM table_1; -- concatenation avec ||
SELECT SUBSTRING(col1, 1, 5) FROM table_1; -- extrait 5 caracteres depuis pos 1 (les 5 premiers caractères)
SELECT REPLACE(col1, 'e', 'E') FROM table_1; -- remplace e par E
SELECT SPLIT_PART(col2, '@', 2) FROM table_1; -- partie apres @
SELECT LEFT(col1, 3) FROM table_1; -- 3 premiers caracteres
SELECT RIGHT(col1, 3) FROM table_1; -- 3 derniers caracteres

```

5.2 Fonctions sur les dates

COMMANDE	DESCRIPTION
NOW()	DATE ET HEURE ACTUELLE
CURRENT_DATE	DATE DU JOUR
EXTRACT('YEAR' FROM col)	EXTRAIRE ANNEE
EXTRACT('MONTH' FROM col)	EXTRAIRE MOIS
EXTRACT(DOW FROM col)	EXTRAIRE JOUR DE LA SEMAINE (0 = DIMANCHE)
DATE_TRUNC('MONTH', col)	TRONQUER AU MOIS
DATE_TRUNC('YEAR', col)	TRONQUER A L'ANNEE
INTERVAL	AJOUTER/SUPPRIMER DES JOURS
::TEXT	CONVERTIR DATE EN TEXT
TO_CHAR	FORMATER

```

SELECT NOW(); -- date et heure actuelles
SELECT CURRENT_DATE; -- date du jour
SELECT EXTRACT(YEAR FROM col) FROM table_1; -- annee
SELECT EXTRACT(MONTH FROM col) FROM table_1; -- mois
SELECT EXTRACT(DOW FROM col) FROM table_1; -- jour semaine (0=dim)
SELECT DATE_TRUNC('month', col) FROM table_1; -- tronquer au mois
SELECT DATE_TRUNC('year', col) FROM table_1; -- tronquer a l'annee
SELECT col - INTERVAL '7 days' FROM table_1; -- ajouter 7 jours
SELECT AGE(col) FROM table_1; -- age depuis aujourd'hui
SELECT date_commande::TEXT FROM table_1; -- convertir en texte
SELECT TO_CHAR(col, 'DD/MM/YYYY') FROM table_1; -- formater TO

```

5.3 Fonctions sur les dates

COMMANDE	DESCRIPTION
CASE WHEN	FONCTION CONDITIONNELLE (IF,ELIF,ELSE)
COALESCE	REPLACE LES NULL PAR UNE VALEUR

NULLIF	REPLACE UNE VALEUR PAR UN NULL

```

select col1,col2,
case
  when col2 < 20 then 'cas 1'
  when col2 < 100 then 'cas 2'
  else 'cas 3'
end as nom_col_creee
from table_1
-- ===== coalesce =====
select col1, coalesce(col2,'valeur de remplacement') as alias_col2
from table_1
-- ===== NULLIF =====
select col1,col2,
nullif (col1,0) as alias_col1 from table_1

```

Q46	[N2]	Afficher les produits dont le prix est superieur au prix moyen de leur categorie.
		Indice : Sous-requete correelee
Q47	[N2]	Afficher les clients qui n'ont passe aucune commande depuis 2022.
		Indice : NOT IN ou NOT EXISTS
Q48	[N2]	Avec une CTE, calculer le CA par client puis afficher ceux dont le CA depasse 1000 euros.
Q49	[N2]	Creer une vue v_catalogue affichant nom, categorie, prix et stock de tous les produits en stock.
Q50	[N2]	Avec une CTE, trouver le mois ayant genere le plus de CA.
Q51	[N2]	Afficher les produits commandes par au moins 3 clients differents.
		Indice : COUNT(DISTINCT client_id)
Q52	[N3]	Avec 2 CTEs : CA par categorie, puis afficher uniquement les categories dans le top 3.
Q53	[N2]	Creer une vue v_top_clients affichant les 10 clients avec le plus grand nombre de commandes.
Q54	[N3]	Afficher les commandes dont le total est superieur a la moyenne des commandes du meme mois.
		Indice : Sous-requete correelee avec DATE_TRUNC
Q55	[BONUS]	Avec une CTE recursive (optionnel avance) : generer une suite de nombres de 1 a 10.

Q56	[N1]	Afficher les emails des clients en majuscules.
Q57	[N2]	Afficher le nom du produit et le domaine de l'email du client dans la meme requete (JOIN + SPLIT_PART).
Q58	[N2]	Afficher les commandes passees un lundi.
		Indice : <code>EXTRACT(DOW ...) = 1</code>
Q59	[N1]	Categoriser les produits en 'Pas cher' (<20), 'Raisnable' (<100), 'Cher' avec CASE WHEN.
Q60	[N1]	Afficher les clients avec leur ville, en remplaçant les NULL par 'Ville inconnue'.
Q61	[N2]	Afficher le CA par annee et par mois sous le format 'YYYY-MM'.
		Indice : <code>TO_CHAR</code>
Q62	[N2]	Afficher le nombre de jours ecoules depuis chaque commande.
		Indice : <code>CURRENT_DATE - date_commande</code>
Q63	[N2]	Afficher les produits dont le nom contient un chiffre.
		Indice : <code>SIMILAR TO</code> ou <code>regexp_match</code>
Q64	[N2]	Afficher le CA total pour les commandes du dernier trimestre.
		Indice : <code>DATE_TRUNC + INTERVAL</code>
Q65	[N2]	Afficher les initiales de chaque client (premiere lettre du prenom + premiere lettre du nom).
		Indice : <code>SPLIT_PART + LEFT</code>